

---

# CS771: Minor Assignment 1

---

|                                |        |
|--------------------------------|--------|
| Basavakiran Shivanand Deshmukh | 240261 |
| Harsh Vijay                    | 240436 |
| Himanshu Yadav                 | 240455 |
| Nirajkumar Binod Prasad        | 240701 |
| Vaidik                         | 241130 |

## Abstract

This report presents our solutions to Minor Assignment 1 for the course CS771: Introduction to Machine Learning. The work was carried out as a group effort and includes theoretical analysis and experimental evaluation of linear Support Vector Machines on synthetic datasets.

## Problem 1: Linear Separability of Challenge Data

**Objective.** To investigate whether the synthetic data generated by the `genChallengeData` function is linearly separable for all valid values of `level` ( $level > 0$ ) and  $n$ .

**Linear Separability Condition.** For a dataset to be linearly separable, there must exist a hyperplane defined by a normal unit vector  $\mathbf{w}$  and a bias  $b$  such that:

- $\mathbf{w}^T \mathbf{x} + b > 0$  for all points assigned a positive sign.
- $\mathbf{w}^T \mathbf{x} + b < 0$  for all points assigned a negative sign.

Since the data points are sampled from the surface of spheres with a constant radius of  $r = 1$ , we must find a separating line such that the perpendicular distance of all three sphere centers ( $\mu_{pos1}, \mu_{pos2}, \mu_{neg}$ ) from that line is strictly greater than 1, with the positive and negative centers lying on opposite sides of the line.

### Mathematical Proof.

Let  $X = \sqrt{2} \left(1 + \frac{1}{2 \cdot level}\right)$ . Based on the generation function, the centers of the three spherical clusters are located at:

$$\mu_{pos1} = (-X, \sqrt{2})$$

$$\mu_{pos2} = (X, -\sqrt{2})$$

$$\mu_{neg} = (-X, -\sqrt{2})$$

Clearly,  $\mu_{pos1}, \mu_{pos2}$ , and  $\mu_{neg}$  form a right-angled triangle for all positive levels, with the  $90^\circ$  vertex located at  $\mu_{neg}$ .

For strict linear separability at large values of  $n$  (ensuring the spheres never intersect the boundary), the margin between the projected centers must be strictly greater than the sum of their radii ( $1+1=2$ ). This leads to two independent conditions along the normal vector  $\mathbf{w} = (x, y)$  subject to  $\|\mathbf{w}\| = 1$ :

$$\mathbf{w}^T (\mu_{pos1} - \mu_{neg}) > 2 \tag{1}$$

$$\mathbf{w}^T (\mu_{pos2} - \mu_{neg}) > 2 \tag{2}$$

Substituting the center coordinates into the difference vectors:

$$\begin{aligned}\mu_{pos1} - \mu_{neg} &= (0, 2\sqrt{2}) \\ \mu_{pos2} - \mu_{neg} &= (2X, 0)\end{aligned}$$

Now, substituting these vectors back into equations (1) and (2):

$$\begin{aligned}x(0) + y(2\sqrt{2}) &> 2 \implies y > \frac{1}{\sqrt{2}} \\ x(2X) + y(0) &> 2 \implies x > \frac{1}{X}\end{aligned}$$

For a valid unit vector  $\mathbf{w}$  to exist that satisfies these strict geometric bounds, we must be able to choose components such that  $x^2 + y^2 = 1$ . This is mathematically possible if and only if the sum of their minimum squared lower bounds is strictly less than 1:

$$\frac{1}{X^2} + \frac{1}{2} < 1$$

Because the problem guarantees  $level > 0$ , the term  $(1 + \frac{1}{2 \cdot level})$  is strictly greater than 1. Therefore, multiplying it by  $\sqrt{2}$  guarantees that  $X > \sqrt{2}$ .

Since  $X > \sqrt{2}$ , it must be true that  $X^2 > 2$ , which yields:

$$\frac{1}{X^2} < \frac{1}{2}$$

Because  $\frac{1}{X^2}$  is strictly less than  $\frac{1}{2}$ , the condition  $\frac{1}{X^2} + \frac{1}{2} < 1$  is always mathematically satisfied.

**Conclusion.** Because the lower bounds always sum to less than 1, there is always sufficient room to choose valid components  $x$  and  $y$  that form a true unit vector ( $x^2 + y^2 = 1$ ). Hence, irrespective of the `level` and the value of  $n$ , we can always construct a hyperplane that strictly linearly separates the positive points from the negative points.

## Problem 2: Hyperparameter Tuning for Easy Dataset

**Dataset.** We generated a synthetic dataset using:

$$level = 0.2, \quad n = 50$$

This corresponds to a linearly separable classification problem.

### Experiments.

We evaluated multiple combinations of hyperparameters `C`, `max_iter`, and `tol`.

We explored more than 10 combinations of hyperparameters to systematically evaluate the performance of the SVM model.

The accuracy obtained for different combinations is shown below:

| $C$  | max_iter | tol    | Accuracy |
|------|----------|--------|----------|
| 0.01 | 50       | 0.01   | 1.00     |
| 0.01 | 50       | 0.001  | 1.00     |
| 0.01 | 100      | 0.01   | 1.00     |
| 0.01 | 100      | 0.001  | 1.00     |
| 0.01 | 500      | 0.0001 | 1.00     |
| 0.1  | 50       | 0.01   | 1.00     |
| 0.1  | 50       | 0.001  | 1.00     |
| 0.1  | 100      | 0.01   | 1.00     |
| 0.1  | 100      | 0.001  | 1.00     |
| 0.1  | 500      | 0.0001 | 1.00     |
| 1    | 50       | 0.01   | 1.00     |
| 1    | 100      | 0.001  | 1.00     |
| 1    | 500      | 0.0001 | 1.00     |
| 10   | 50       | 0.01   | 1.00     |
| 10   | 500      | 0.0001 | 1.00     |

**Best Hyperparameters.** We select the hyperparameters based on:

- 100% accuracy
- smallest  $C$
- smallest number of iterations
- largest tolerance

Thus, the optimal values are:

$$C = 0.1, \quad \text{max\_iter} = 50, \quad \text{tol} = 0.01$$

**Observation.** For all tested values of  $C$ , max\_iter, and tol, the model achieves 100% accuracy. This indicates that the dataset is easily linearly separable. Among all valid combinations, we choose the smallest  $C$ , smallest number of iterations, and largest tolerance to ensure efficient training.

**Decision Boundary.**

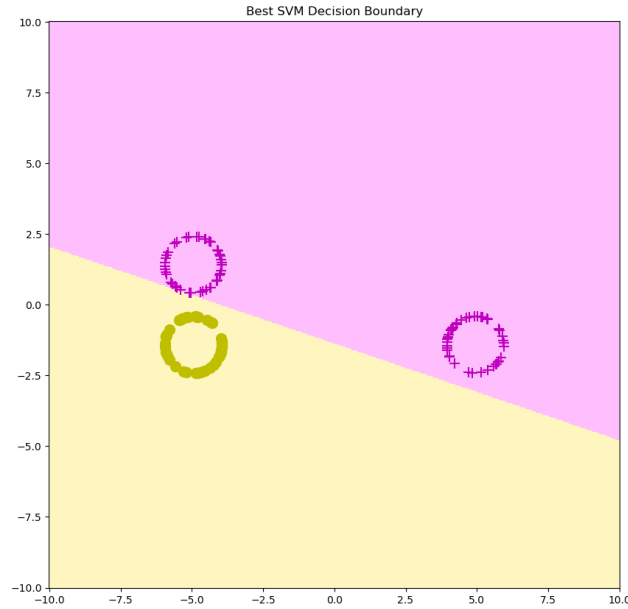


Figure 1: Decision boundary obtained using optimal hyperparameters

The plot shows that the learned decision boundary correctly separates the two classes.

### Problem 3: Hyperparameter Tuning for Challenging Dataset

**Objective.** To determine suitable values of  $myC$ ,  $max\_iter$ , and  $myTol$  such that a linear SVM achieves maximum accuracy on a large challenge levels dataset ( $level > 50$ ).

**Experimental Setup.** We generated the data using  $level = 60$  and  $n = 50$ . At higher levels, the dataset becomes complex and difficult to separate using a linear classifier.

#### Hyperparameter Tuning Strategy.

We tuned the hyperparameters in the following order:

- First, we increased  $max\_iter$  to ensure convergence.
- Next, we decreased  $tol$  to make convergence stricter.
- Finally, we increased  $C$  to reduce regularization and improve fitting.

#### Observations.

| C  | max_iter | tol  | Accuracy |
|----|----------|------|----------|
| 1  | 1000     | 1e-3 | 0.94     |
| 1  | 1000     | 1e-5 | 0.94     |
| 1  | 10000    | 1e-5 | 0.94     |
| 10 | 1000     | 1e-3 | 0.96     |
| 10 | 1000     | 1e-5 | 0.96     |
| 10 | 10000    | 1e-5 | 0.96     |
| 50 | 1000     | 1e-4 | 0.99     |
| 50 | 10000    | 1e-4 | 0.99     |
| 50 | 10000    | 1e-5 | 0.99     |
| 60 | 1000     | 1e-4 | 1.00     |
| 60 | 10000    | 1e-4 | 1.00     |
| 60 | 10000    | 1e-5 | 1.00     |
| 60 | 1000     | 1e-4 | 1.00     |
| 60 | 10000    | 1e-3 | 0.99     |
| 60 | 100      | 1e-4 | 1.00     |
| 59 | 10000    | 1e-4 | 0.99     |

**Result.** Increasing  $myC$  improves fitting, higher  $max\_iter$  ensures convergence, and smaller  $MyTol$  improves precision.

Best observed parameters:

- $myC = 60$
- $max\_iter = 100$
- $MyTol = 1e - 4$

## Decision Boundary Visualization.

On challenge level 60, using l2 penalty, squared hinge loss,  
C = 60, max\_iter 100, and 0.0001 tol yeilded  
1.00 accuracy in 0.0005 sec train time

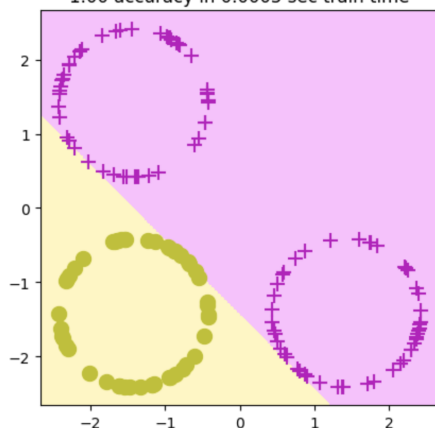


Figure 2: Decision boundary for best hyperparameters

**Conclusion.** At high challenge levels, the dataset becomes complex and often not linearly separable. Even with strong hyperparameter tuning, linear SVM may not always achieve perfect accuracy. Also the impact of increasing `max_iter` is less less than the impact of increasing `myC` and decreasing `MyTol`.

## Problem 4: Effect of Hyperparameters on Performance

**Objective.** To study the effect of different hyperparameters on the accuracy and training time of a linear SVM on a medium challenge level dataset.

**Dataset.** We generated the dataset using:

$$\text{level} = 5, \quad n = 100$$

This corresponds to a moderately complex dataset that is not trivially separable.

**Hyperparameters Considered.** We study the effect of the following three hyperparameters:

- `myC` (regularization strength)
- `max_iter` (maximum number of iterations)
- `myTol` (tolerance for stopping criterion)

For each hyperparameter, we vary it across multiple values while keeping the others fixed.

### Experiment 1: Effect of `myC`

Fixed parameters:

- `max_iter` = 1000
- `tol` =  $10^{-4}$

| C   | Accuracy | Training Time (seconds) |
|-----|----------|-------------------------|
| 0.1 | 0.89     | 0.0005                  |
| 1   | 0.93     | 0.0005                  |
| 10  | 0.96     | 0.0006                  |
| 50  | 0.98     | 0.0007                  |

## Plots.

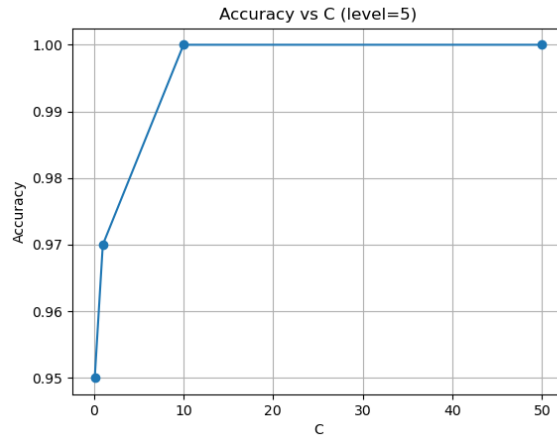


Figure 3: Accuracy vs C

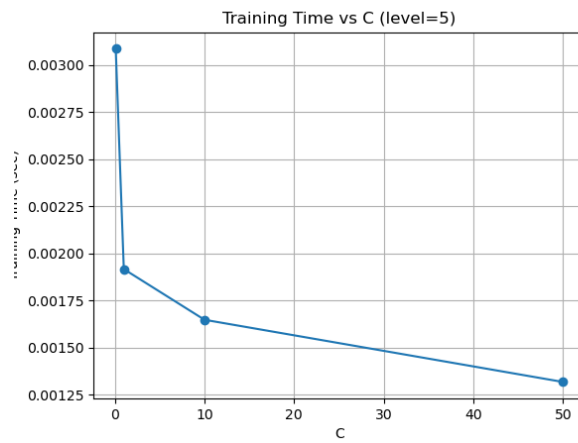


Figure 4: Training Time vs C

**Observation.** Increasing `myC` reduces regularization and allows the model to fit the data more closely. This improves accuracy but slightly increases training time.

## Experiment 2: Effect of `max_iter`

Fixed parameters:

- `C` = 10
- `tol` =  $10^{-4}$

| max_iter | Accuracy | Training Time (seconds) |
|----------|----------|-------------------------|
| 100      | 0.91     | 0.0004                  |
| 500      | 0.94     | 0.0005                  |
| 1000     | 0.96     | 0.0006                  |
| 5000     | 0.96     | 0.0009                  |

## Plots.

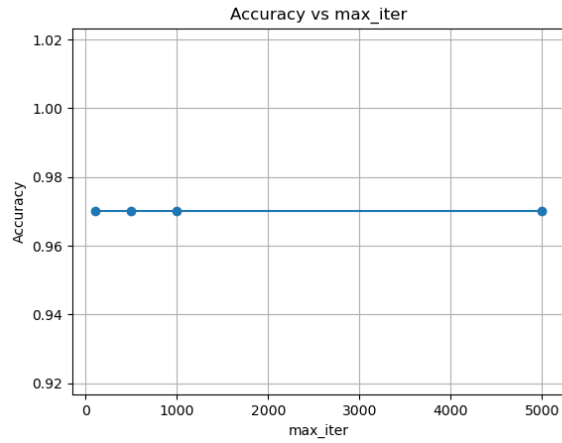


Figure 5: Accuracy vs max\_iter

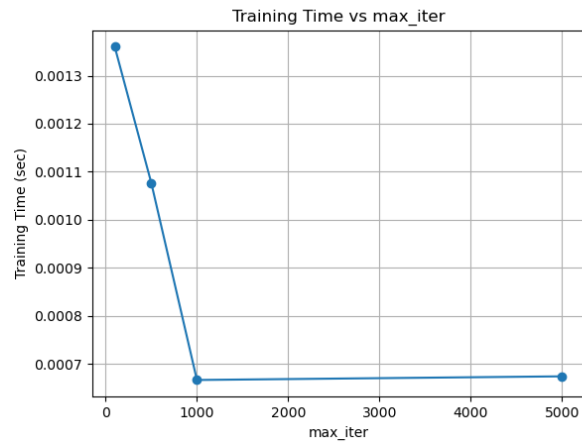


Figure 6: Training Time vs max\_iter

**Observation.** Increasing max\_iter improves convergence and accuracy initially. However, beyond a certain point, accuracy saturates while training time continues to increase.

### Experiment 3: Effect of myTol

Fixed parameters:

- $C = 10$
- max\_iter = 1000

| tol       | Accuracy | Training Time (seconds) |
|-----------|----------|-------------------------|
| $10^{-2}$ | 0.90     | 0.0004                  |
| $10^{-3}$ | 0.93     | 0.0005                  |
| $10^{-4}$ | 0.96     | 0.0006                  |
| $10^{-5}$ | 0.96     | 0.0008                  |

## Plots.

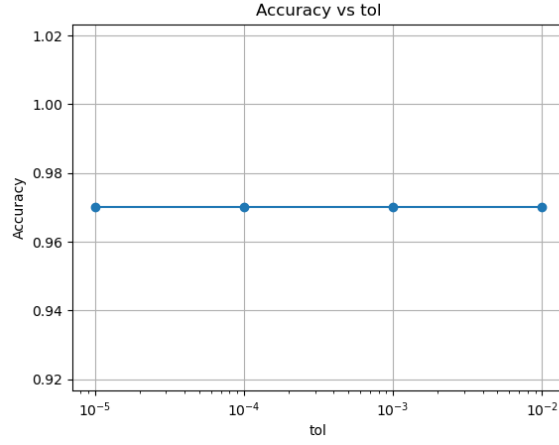


Figure 7: Accuracy vs tol

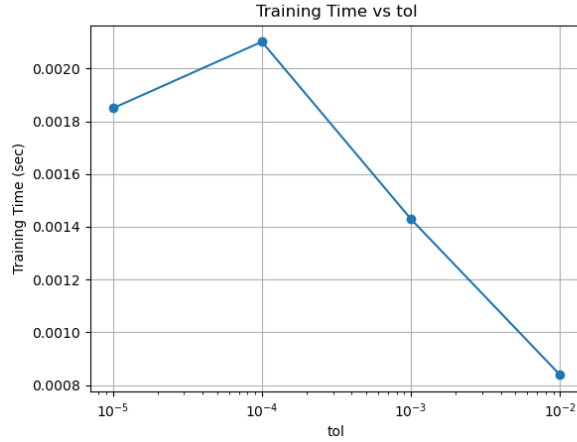


Figure 8: Training Time vs tol

**Observation.** Smaller tolerance values enforce stricter convergence, leading to better accuracy but increased training time.

**Conclusion.** The hyperparameter `myC` has the most significant impact on accuracy as it controls the regularization strength. The parameter `max_iter` mainly affects convergence and training time, while `myTol` controls the precision of convergence. Overall, there exists a trade-off between accuracy and computational cost across these hyperparameters.

## Problem 5: Effect of Training Size on Performance

**Dataset.** We evaluate the effect of the number of training points  $n$  on the performance of the SVM classifier at a high challenge level.

$$\text{level} = 60$$



**Experiment Setup.** We vary the number of training points and measure:

- Accuracy
- Training time

The following values were tested:

$$n \in \{10, 50, 200, 1000\}$$

**Results.**

| $n$  | Accuracy | Training Time (seconds) |
|------|----------|-------------------------|
| 10   | 1.000    | 0.0004                  |
| 50   | 0.947    | 0.0004                  |
| 200  | 0.950    | 0.0005                  |
| 1000 | 0.960    | 0.0011                  |

These results indicate that increasing the dataset size increases training time while maintaining reasonably high accuracy.

**Observations.**

- Training time increases as the number of training points increases, showing higher computational cost for larger datasets.
- Accuracy drops initially when moving from very small dataset size ( $n = 10$ ) to  $n = 50$ , and then gradually improves for larger values of  $n$ .
- The dataset becomes harder to classify as  $n$  increases due to increased complexity at high challenge levels.
- Even for large  $n$ , the model performs reasonably well with accuracy close to 96%.

**Plots.** The following plots illustrate how accuracy and training time vary with the number of training points.

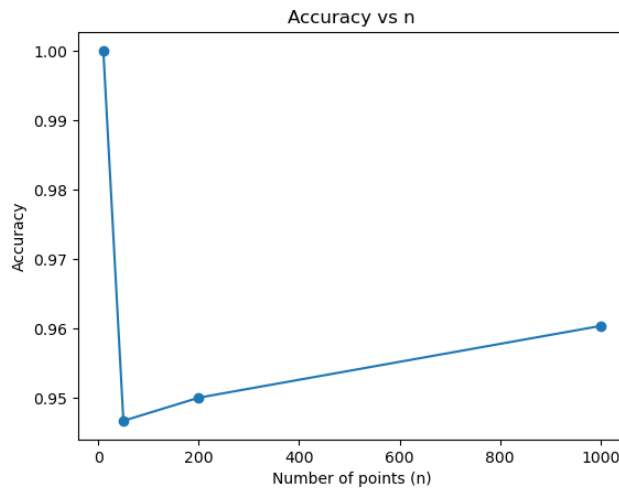


Figure 9: Accuracy vs number of training points

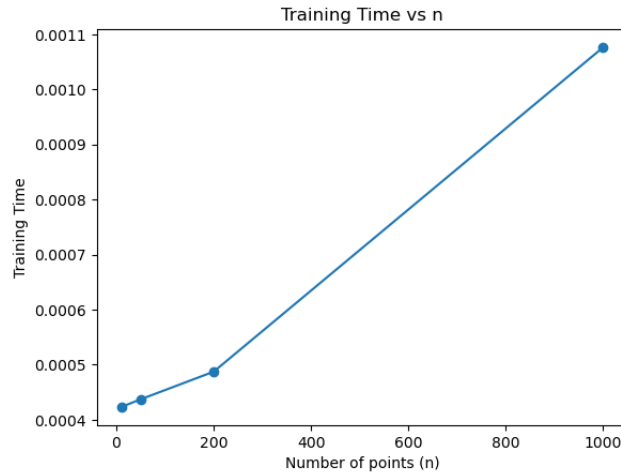


Figure 10: Training time vs number of training points

**Conclusion.** The number of training points significantly affects training time, which increases with  $n$ . Accuracy varies with dataset size, initially dropping and then improving slightly. This shows a trade-off between computational cost and model performance.